

자료구조및실습: 2018년 1학기

제 1 주:

객체의 표현

마 방 진



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



© J.-H. Kang, CNU

"마방진 이란?"



□ 마방진 (Magic Square)

■ 주어진 조건

- 가로 세로의 크기가 동일한 정사각형 판이 주어져 있다.
- 한 변의 크기를 마방진 문제의 차수(order)라고 한다.
- 차수는 3보다 크거나 같은 홀수이다.

■ 문제

- 차수가 N일 때, 1부터 $N*N$ 까지의 정수를 한번씩 모두 사용하여 판을 채우되 가로, 세로, 대각선의 합이 모두 동일하게 채워진 판을 찾는다.

- 예: 차수(order)가 3인 마방진

- ◆ 1부터 $3*3(9)$ 까지의 정수를 한번씩 사용하여, 판을 채운다.
- ◆ 가로, 세로, 대각선의 합이 모두 15이다.

	[0]	[1]	[2]
[0]	8	1	6
[1]	3	5	7
[2]	4	9	2

출발점

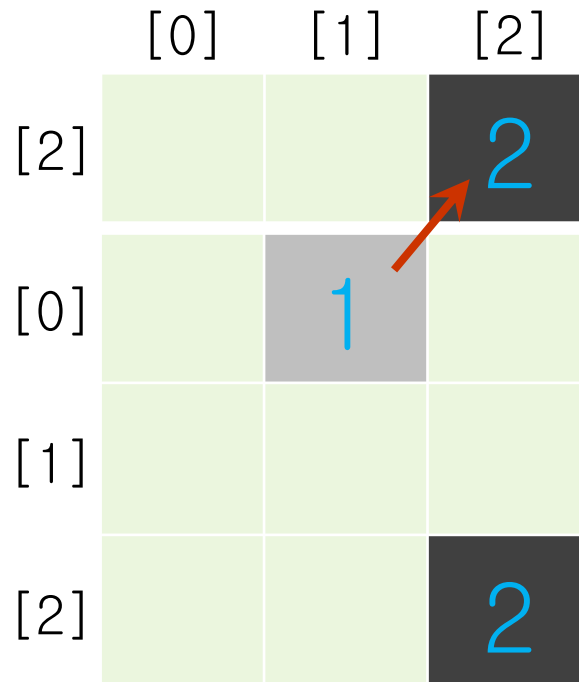
- 맨 윗 줄의 한 가운데:
 - 차수가 3인 경우, [0][1]의 위치

	[0]	[1]	[2]
[0]		1	
[1]			
[2]			

마방진이란?

□ 다음 채울 곳

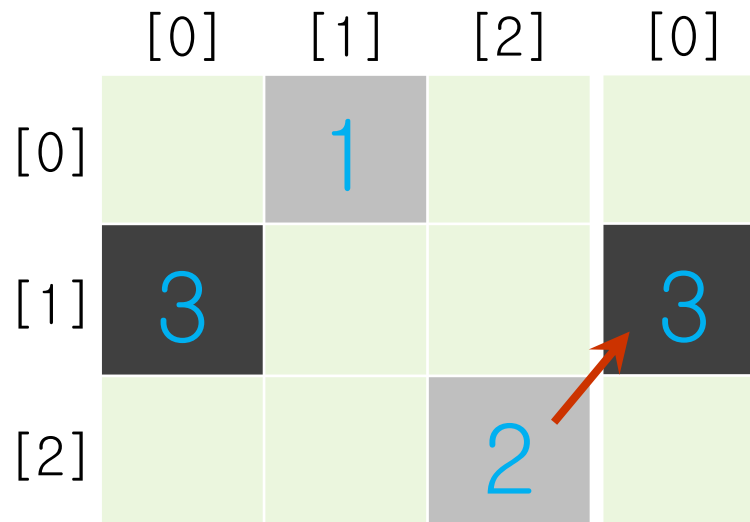
- 항상 **오른쪽 위**로 올라간다.
- 판을 벗어나게 되면?
 - 1을 넣은 위치에서 오른쪽 위로 올라가면 판의 위쪽으로 벗어난다.
 - 판을 위아래가 붙은 마치 원통형인 것처럼 생각하면, 그 다음 위치는 [2][2]가 된다.



마방진이란?

□ 다음 채울 곳

- 항상 **오른쪽 위**로 올라간다.
- [2][2]위치에서 오른쪽 위로 올라가면 판의 오른쪽으로 벗어난다.
 - 판을 좌우로 붙은 원통형인 것처럼 생각하면, 그 다음 위치는 [1][0]가 된다.



마방진이란?

□ 다음 채울 곳이 이미 채워져 있으면?

■ 다음 위치는?

- '3'을 채운 [1][0]의 오른쪽 위 위치인 [0][1]에는 이미 '1'이 채워져 있다.

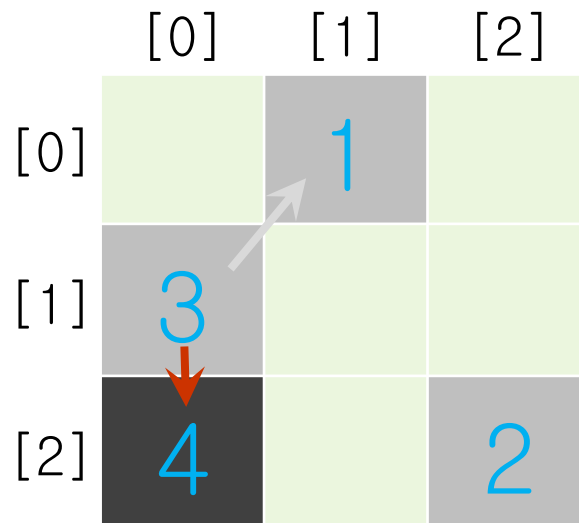
	[0]	[1]	[2]
[0]		1	
[1]	3		
[2]			2

마방진이란?

□ 다음 채울 곳이 이미 채워져 있으면?

■ 바로 한 칸 아래가 다음 위치이다.

- '3'을 채운 [1][0]의 오른쪽 위 위치인 [0][1]에는 이미 '1'이 채워져 있다.
- 그러므로 '4'를 채울 다음 위치는 [2][0]이다.



마방진이란?

□ 다음 채울 곳

■ 오른쪽 위로 가자!

- 다음 위치 [1][1]은 비어있다.
- 이곳에 '5'를 채운다.

	[0]	[1]	[2]
[0]		1	
[1]	3	5	
[2]	4		2

마방진이란?

□ 다음 채울 곳

■ 오른쪽 위로 가자!

- 다음 위치 [0][2]는 비어있다.
- 이곳에 '6'을 채운다.

	[0]	[1]	[2]
[0]		1	6
[1]	3	5	
[2]	4		2

마방진이란?

□ 다음 채울 곳

■ 오른쪽 위로 가자!

- 상하좌우가 붙어있다고 생각하면, 다음 위치는 [2][0].
- 이곳은 이미 '4'로 채워진 곳.

	[0]	[1]	[2]	
[0]		1	6	[0]
[1]	3	5		[1]
[2]	4		2	[2]

	[0]	[1]	[2]
[0]		1	6
[1]	3	5	
[2]	4		2

□ 다음 채울 곳

■ 채워져 있으므로 바로 아래로!

- 상하좌우가 붙어있다고 생각하면, 다음 위치는 [2][0].
- 이곳은 이미 '4'로 채워진 곳.
- 다음 위치는 바로 한 칸 아래인 [1][2]



□ 다음 채울 곳

- 오른쪽 위로 가자!
 - 다음 위치는 [0][0].
 - 이곳에 '8'을 채운다.

	[0]	[1]	[2]	[0]
[0]	8	1	6	8
[1]	3	5	7	3
[2]	4		2	4

마방진이란?

□ 다음 채울 곳

- 오른쪽 위로 가자!
 - 다음 위치는 [2][1].
 - 이곳에 '9'를 마지막으로 채운다.

	[0]	[1]	[2]
[2]	4	9	2
[0]	8	1	6
[1]	3	5	7
[2]	4	9	2

마방진이란?

□ 채우기 종료!

- '1' 부터 '9' 까지의 정수를 모두 채웠다.
- 가로, 세로, 대각선의 합은 모두 동일하게 15 이다.

	[0]	[1]	[2]
[0]	8	1	6
[1]	3	5	7
[2]	4	9	2

마방진이란?



□ 요약

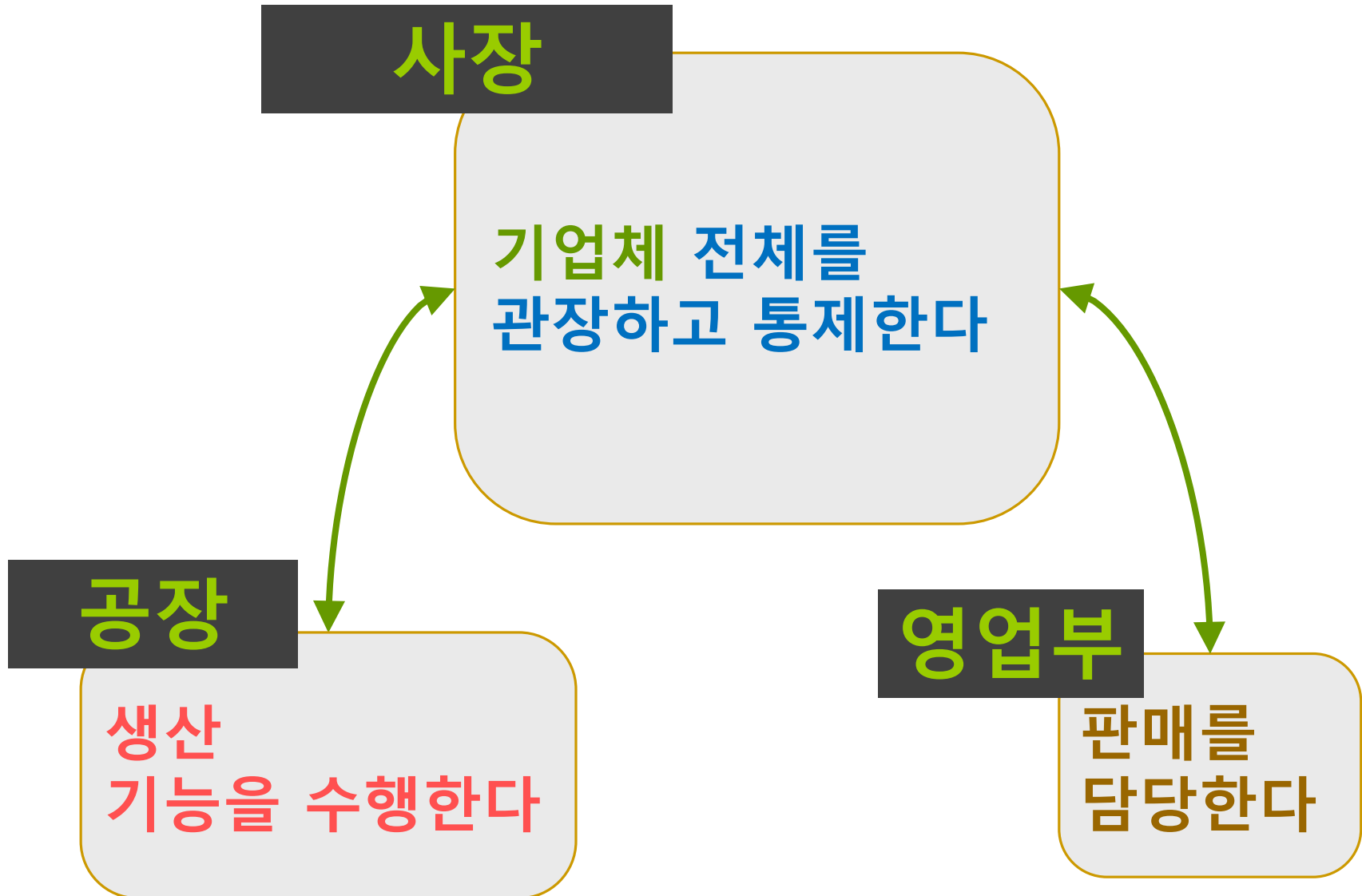
- 차수가 홀수이면 항상 해결 가능
- 차수가 짝수이면?

- 문제의 인식, 그리고 알고리즘으로, 프로그램으로!
 - 문제의 인식
 - ◆ 문제를 분석하고 이해
 - 알고리즘으로!
 - ◆ 규칙의 발견, 그리고 정확히 표현
 - 프로그램으로!
 - ◆ 주어진 언어 (우리는 Java 언어)로 적절하게 표현
 - ◆ 객체의 인식

"Model-View-Controller"



□ App 에서의 역할 구분



□ App 에서의 역할 구분

Controller

App 전체를
관장하고 통제한다

Model

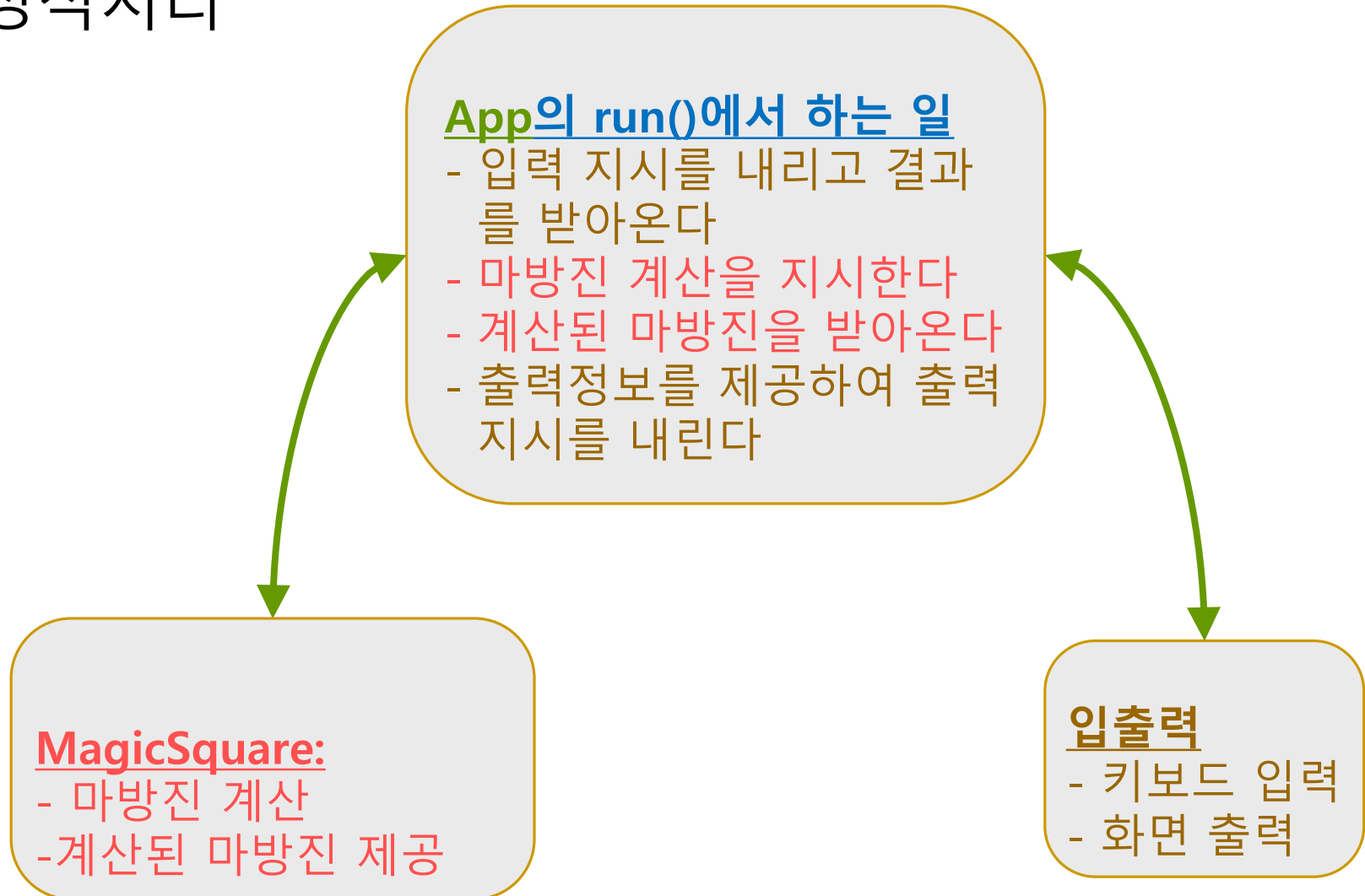
핵심 두뇌 (생산)
기능을 수행한다

View

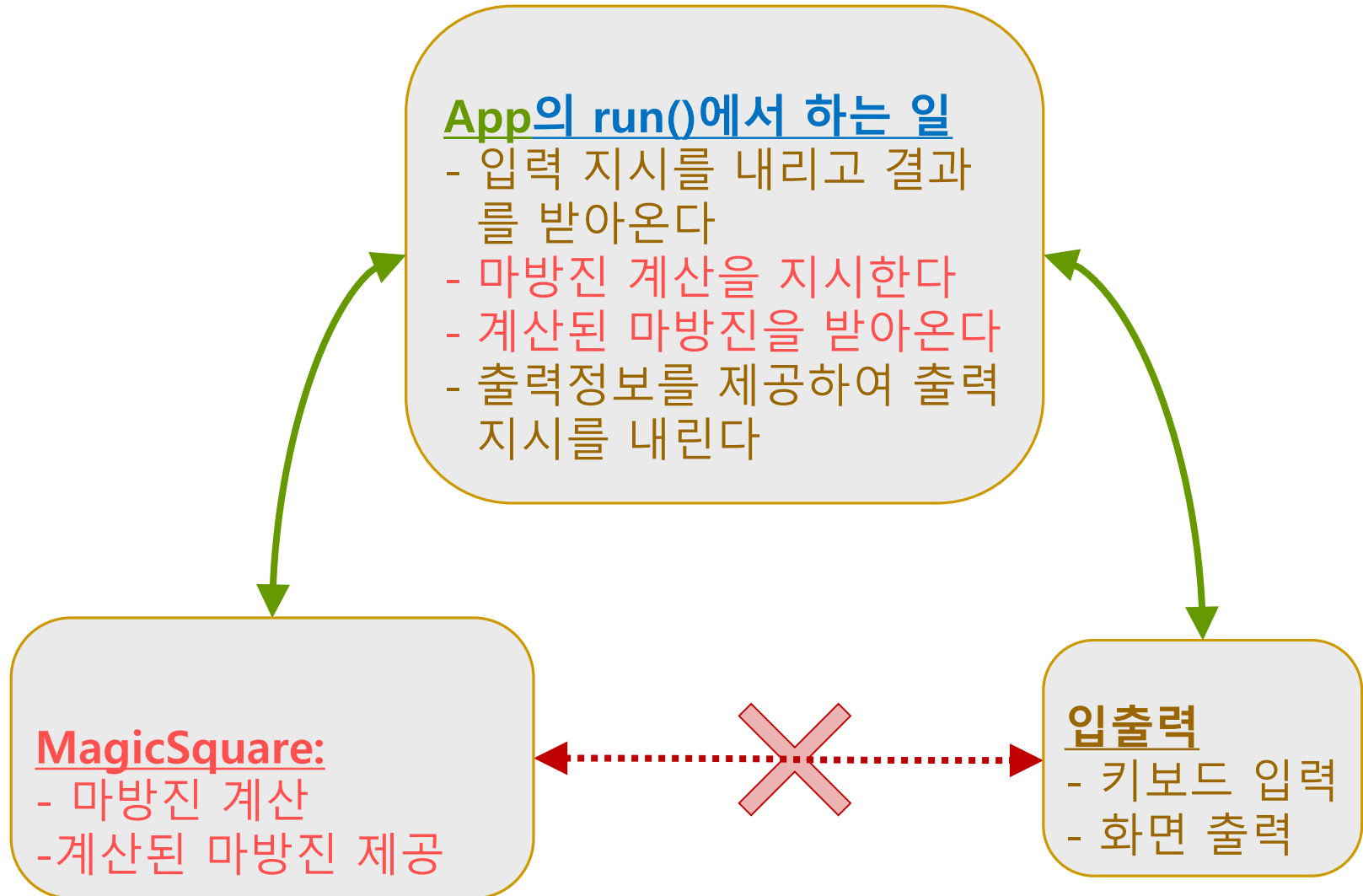
입출력을
담당한다

□ 모든 프로그램은 MVC 형태로

■ 성적처리



□ 부하 직원이 상사 몰래?



□ MVC

Controller

App의 run()에서 하는 일

- 입력 지시를 내리고 결과를 받아온다
- 마방진 계산을 지시한다
- 계산된 마방진을 받아온다
- 출력정보를 제공하여 출력 지시를 내린다

Model

MagicSquare:

- 마방진 계산
- 계산된 마방진 제공

View

입출력

- 키보드 입력
- 화면 출력

□ 각각을 class 로!

■ 마방진 프로그램

- Controller: Class "AppController"
- Model: Class "MagicSquare"
- View: Class "AppView"

"for 문"

□ "for" 문

■ while과 동일한 기능

```
int sum = 0 ;  
int i = 0 ;  
while ( i < maxNumber ) {  
    sum = sum + i ;  
    i++ ;  
}
```

```
int sum = 0 ;  
  
for ( int i = 0 ; i < maxNumber ; i++ ) {  
    sum = sum + i ;  
}
```

과제에서 해결할 문제



문제

- 주어진 차수에 합당한 마방진 문제를 해결한다.

입출력

입력

- 매번 마방진 차수를 입력 받는다:
- 음수이면 "<마방진 풀이를 종료합니다>" 라는 메시지를 내보내고 프로그램을 종료한다
- 음수가 아니면 차수의 오류 검사를 다음과 같이 한다:
 - ◆ 차수가 3 보다 작으면 "오류: 차수가 너무 작습니다. 3 보다 크거나 같아야 합니다." 라는 메시지를 내보낸다.
 - ◆ 차수가 99 보다 크면 "오류: 차수가 너무 큼니다. 99 보다 작거나 같아야 합니다." 라는 메시지를 내보낸다.
 - ◆ 차수가 짝수 이면 "오류: 차수가 짝수입니다. 홀수이어야 합니다." 라는 메시지를 내보낸다.

출력

- 매번 입력된 차수에 대한 마방진 풀이 결과를 출력한다.

출력의 예

<<< 마방진 풀이를 시작합니다 >>>

마방진 차수를 입력하시오 (음수를 입력하면 종료합니다): **2**
오류: 차수가 너무 작습니다. 3 보다 크거나 같아야 합니다.

마방진 차수를 입력하시오 (음수를 입력하면 종료합니다): **5**
Magic Square Board: Order 5

	[0]	[1]	[2]	[3]	[4]
[0]	17	24	1	8	15
[1]	23	5	7	14	16
[2]	4	6	13	20	22
[3]	10	12	19	21	3
[4]	11	18	25	2	9

마방진 차수를 입력하시오(음수를 입력하면 종료합니다): **6**
오류: 차수가 짝수입니다. 홀수이어야 합니다.

마방진 차수를 입력하시오(음수를 입력하면 종료합니다): **100**
오류: 차수가 너무 큼니다. 99 보다 작거나 같아야 합니다.

마방진 차수를 입력하시오(음수를 입력하면 종료합니다): **3**
Magic Square Board: Order 3

	[0]	[1]	[2]
[0]	8	1	6
[1]	3	5	7
[2]	4	9	2

마방진 차수를 입력하시오(음수를 입력하면 종료합니다): **7**
Magic Square Board: Order 7

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	30	39	48	1	10	19	28
[1]	38	47	7	9	18	27	29
[2]	46	6	8	17	26	35	37
[3]	5	14	16	25	34	36	45
[4]	13	15	24	33	42	44	4
[5]	21	23	32	41	43	3	12
[6]	22	31	40	49	2	11	20

마방진 차수를 입력하시오: **-1**

<<< 마방진 풀이를 종료합니다 >>>

이 과제에서 필요한 class 는?

- class ApplicationController

- class AppView

- Model

- class MagicSquare

- class Board

- class CellLocation

- enum OrderValidity

- enum MessageID

프로그램이 시작하는 곳을 포함하는 특별한 class



□ "main()" 을 포함하는 특별한 class

```
public class DS1_01_학번_이름 {  
    public static void main (String[] args)  
    {  
        ApplicationController appController = new ApplicationController() ;  
        // ApplicationController 가 실질적인 main class 이다.  
        appController.run() ;  
        // 여기 main()에서는 앱 실행이 시작되도록 해주는 일이 전부이다.  
    }  
}
```


□ main()은 어떻게?

- main() 을 위한 class에서는 App에서 필요로 하는 어떠한 변수 (또는 상수)도 선언하지 않는다.
- 이제 실제 main 의 역할로서 필요하다고 판단되는 모든 것은 class "AppController" 안에 선언한다.
- main() 함수는 단지 아래의 코드만 필요하다.

```
public static void main (String[] args)
{
    AppController appController = new AppController() ;
    appController.run() ; // 여기에서는 앱이 실행되도록 해주는 일이 전부이다.
}
```

- 앞으로 모든 Java 프로그램은 이런 형태로 작성한다.
 - Java 프로그램 뿐만 아니라 어떤 언어로 된 프로그램도 이와 같이 작성한다.
 - MVC 모델도 철저히 지킨다.

Class “AppController”

□ ApplicationController: 생성자, 공개함수

■ 생성자

- `public ApplicationController() ;`

■ 공개함수

- `public void run() ; // 유일한 공개함수`

Class “AppController”의 구현

□ ApplicationController: _scanner는 어디에?

```
public class ApplicationController {  
    // 비공개 변수들  
    private AppView _appView ;  
    private Scanner _scanner ;  
    private MagicSquare _magicSquare ;  
  
    // 생성자  
    public ApplicationController()  
    {  
        this._appView = new AppView() ;  
        this._scanner = new Scanner(System.in) ;  
        this._magicSquare = new MagicSquare() ;  
    }  
  
    // 비공개함수의 구현  
    .....  
  
    // 공개함수의 구현  
    .....  
} // End of class "AppController"
```

**_scanner는
AppView 속으로!**

□ ApplicationController: run()

```
public class ApplicationController {
    .....

    // 공개함수의 구현
    public void run()
    {
        this._appView.outputLine("<<< 마방진 풀이를 시작합니다 >>>");
        this._appView.outputLine("");

        int order = this._appView.inputOrder(); // 메시지를 내보내고 차수를 입력 받음
        while (order >= 0) { // 차수가 음수이면 프로그램 종료
            OrderValidity currentOrderValidity = this._magicSquare.checkOrderValidity (order) ;
            if (currentOrderValidity == OrderValidity.Valid ) { // 차수가 유효한지 검사
                this._appView.outputTitleWithOrder (order) ;
                Board solvedBoard= this._magicSquare.solve (order) ;
                // _magicSquare 객체에게 주어진 차수의 마방진을 풀도록 시킨다.
                // 결과로 마방진 판을 얻는다
                this.showBoard (solvedBoard) ; // 마방진을 화면에 보여준다
            }
            else {
                this.showOrderValidityErrorMessage (currentOrderValidity) ;
            }
            order = this._appView.inputOrder() ; // 다음 마방진을 위해 차수를 입력 받음
        }
        this._appView.outputLine("");
        this._appView.outputLine("<<< 마방진 풀이를 종료합니다 >>>");
    }
}
```

□ ApplicationController: showOrderValidityErrorMessage()

```
private void showOrderValidityErrorMessage (OrderValidity anOrderValidity)
{
    switch ( anOrderValidity ) {
        case TooSmall:
            this._appView.outputLine("오류: 차수가 너무 작습니다. 3 보다 크거나 같아야 합니다.");
            break ;
        case TooLarge:
            this._appView.outputLine("오류: 차수가 너무 큼니다. 99 보다 작거나 같아야 합니다. ");
            break ;
        case NotOddNumber:
            this._appView.outputLine("오류: 차수가 짝수입니다. 홀수이어야 합니다.");
            break ;
        default:
            break ;
    }
}
```

□ ApplicationController: showBoard()

```
private void showBoard (Board aBoard) {  
    CellLocation currentLoc = new CellLocation();  
    this.showTitleForColumnIndexes (aBoard.order());  
    for (int row = 0; row < aBoard.order(); row++) {  
        this._appView.outputRowNumber(row);  
        for (int col = 0; col < aBoard.order(); col++) {  
            currentLoc.setRow(row);  
            currentLoc.setCol(col);  
            this._appView.outputCell(aBoard.cell(currentLoc));  
        }  
        this._appView.outputLine("");  
    }  
}
```


□ ApplicationController: showTitleForColumnIndexes()

```
private void showTitleForColumnIndexes (int anOrder) {
    this._appView.outputLine("    "); // 빈칸 6 개
    for (int col = 0 ; col < anOrder; col++) {
        this._appView.output(String.format("%3d", col));
    }
    this._appView.outputLine("");
}
```

양식 "**%3d**" 가 의미하는 문자 배치 형태:
모두 6 칸의 문자열이 만들어진다. (첫번째
문자가 빈칸 임에 유의할 것)



이곳 3 칸에 정수 값이 들어간다

String.format (format, variable_list):

주어진 변수들 (variable_list)을 주어진 양식 (format)에 맞게 문자열 객체를 생성한다. 양식 "**%3d**" 는, 정수 값 하나를 3 칸 안에 배치함을 의미한다. 즉 "%"는 이에 상응하는 변수가 반드시 variable_list 에 주어져야 하며, "**3**" 은 어떤 변수의 값이 문자열 안에서 차지하는 공간을 지정하는 것이며, "**d**"는 상응하는 변수의 값이 정수형으로 문자열 안에 배치됨을 의미한다.

Class “AppView”

□ 공개 함수

```
public AppView() {...} // 생성자
```

```
// 입력 관련 함수들
```

```
public int      inputOrder() {...} // 마방진 차수를 입력 받아 얻는다
```

```
// 출력 관련 함수들
```

```
public void  output (String aString) {...}
```

```
public void  outputLine (String aString) {...}
```

```
public void  outputTitleWithOrder (int anOrder) {...}
```

```
public void  outputRowNumber (int aRowNumber) {...}
```

```
public void  outputCell (int aCellValue) {...}
```

Class “AppView”의 구현

□ AppView: Scanner 선언, 생성자

```
public class AppView {  
    // 비공개 상수/변수들  
    private Scanner    _scanner ;  
  
    // 생성자  
    public AppView ()  
    {  
        this._scanner = new Scanner(System.in) ;  
    }  
  
    // 공개함수의 구현  
    .....  
}
```

□ Class "AppView" 의 구현

```
public class AppView {  
    // 비공개 상수/변수들  
    ....  
    // 생성자  
    .....  
  
    // 공개함수의 구현  
    public int inputOrder () {  
        ..... // 여기를 채우시오  
    }  
}
```

□ Class "AppView" 의 구현

■ 출력 관련 함수들

```
public void outputTitleWithOrder (int anOrder) {  
    .....  
}
```

```
public void outputRowNumber(int aRowNumber)  
{  
    System.out.printf ("[%3d] ", aRowNumber);  
}
```

```
public void outputCell (int aCellValue)  
{  
    System.out.printf (" %3d ", aCellValue);  
}
```

Enum “OrderValidity”

□ Enum "OrderValidity"

```
public enum OrderValidity {  
  
    Valid,  
    TooSmall,  
    TooLarge,  
    NotOddNumber,  
  
} // End of enum "OrderValidity"
```

Class “MagicSquare”



□ MagicSquare: 공개함수

```
public class MagicSquare {  
    .....  
    // 기본 생성자  
    .....  
    // 최대 차수를 사용자가 지정하는 생성자  
    .....  
  
    // 공개함수  
    public OrderValidity checkOrderValidity (int anOrder) {.....}  
        // 주어진 차수가 유효한지를 검사 시킨다.  
        // 검사 결과로 차수의 유효 상태 (OrderValidity) 를 받는다  
  
    public int maxOrder() {.....}  
        // 마방진의 현 상태의 최대 차수를 얻는다  
    public Board solve (int anOrder) {.....}  
        // 주어진 차수의 마방진을 풀도록 시킨다  
        // 차수는 반드시 다시 검사해야 한다.  
        // 풀어서 생성된 마방진 판을 얻는다. 오류가 있으면 null을 얻는다.  
} // End of class "MagicSquare"
```

□ MagicSquare: 생성자

```
public class MagicSquare {  
    private static int DEFAULT_MAX_ORDER = 99 ;  
  
    private int _maxOrder ;  
  
    // 기본 생성자  
    public MagicSquare()  
    {  
        this._maxOrder = MagicSquare.DEFAULT_MAX_ORDER ;  
    }  
  
    // 최대 차수를 사용자가 지정하는 생성자  
    public MagicSquare (int givenMaxOrder)  
    {  
        this._maxOrder = givenMaxOrder ;  
    }  
}
```

여러 가지 생성자를 만들어 놓고,
상황에 따라 골라서 사용할 수도
있다!
모든 생성자의 이름은 동일.

Class “MagicSquare”의 구현



□ MagicSquare: 상수, 인스턴스 변수

```
public class MagicSquare {  
    private static final int DEFAULT_MAX_ORDER = 99 ;  
  
    private int        _maxOrder ;
```

□ MagicSquare: getter

```
public class MagicSquare {  
    private static final int DEFAULT_MAX_ORDER = 99 ;  
  
    private int        _maxOrder ;  
  
    // Getters  
    public int maxOrder() {  
        return this._maxOrder;  
    }  
}
```

□ MagicSquare: 공개함수의 구현

```
public class MagicSquare {  
    private static final int DEFAULT_MAX_ORDER = 99 ;  
  
    private int        _maxOrder ;  
  
    // Getters  
    public int maxOrder() {...}  
  
    // 기본 생성자  
    public MagicSquare()  
    {  
        this._maxOrder = MagicSquare.DEFAULT_MAX_ORDER ;  
    }  
  
    // 최대 차수를 사용자가 지정하는 생성자  
    public MagicSquare (int givenMaxOrder)  
    {  
        this._maxOrder = givenMaxOrder ;  
    }  
}
```


□ 공개함수 "solve()"

```

public Board solve (int anOrder)
{
    if ( this.checkOrderValidity(anOrder) != OrderValidity.Valid) {
        return null ;
    }
    else {
        Board board = new Board (anOrder) ;
        // 차수와 함께 Board 객체 생성자를 call 하여, Board 객체를 생성한다.
        CellLocation currentLoc = new CellLocation (0,this. anOrder/2) ;
        // 출발 위치 (보드의 맨 윗줄 한 가운데)를 현재의 위치로 설정한다.
        CellLocation nextLoc = new CellLocation() ;

        board.setCell (currentLoc, 1) ;    // 보드의 <출발 위치>에 1 을 채운다.

        int lastValue = anOrder * anOrder ;
        for ( int cellValue= 2 ; cellValue <= lastValue ; cellValue++ ) {
            // 단계 1: <현재 위치>로부터 <다음 위치>인 "오른쪽 위" 위치를 계산한다
            ..... // 여기를 채울 것
            // 단계 2: <다음 위치> 가 채워져 있으면
            // <다음 위치>를 <현재 위치>의 바로 한 줄 아래 칸 위치로 수정한다.
            if ( ! board.cellIsEmpty (nextLoc) ) {
                ..... // 여기를 채울 것
            }
            // 단계 3: <다음 위치>를 새로운 <현재 위치>로 한다.
            currentLoc.setRow (nextLoc.row()) ;
            currentLoc.setCol (nextLoc.col()) ;
            // 단계 4: 새로운 <현재 위치>에 number 값을 넣는다.
            board.setCell (currentLoc, cellValue) ;
        }
        return board ;
    }
} // End of solve()

```

Class "CellLocation"



Class CellLocation

- Cell의 위치는 행(row) 과 열(col)로 표현할 수 있다.
- 이 두 값을 하나로 묶어서 객체로 만든다

col

	[0]	[1]	[2]
[0]		1	
[1]			
[2]			

row

□ CellLocation: 생성자, 공개함수

■ 생성자:

- `public CellLocation() ;`
 - ◆ 단지 객체만 생성한다
- `public CellLocation(int givenRow, int givenCol) ;`
 - ◆ 주어진 좌표 (givenRow, givenCol) 을 갖는 객체를 생성한다

■ 공개함수

- `public void setRow (int newRow) ; // setter for row`
- `public int row () ; // getter for row`
- `public void setCol (int newCol) ; // setter for col`
- `public int col () ; // getter for col`

Class “CellLocation”의 구현



□ CellLocation: 인스턴스 변수, 생성자

```
public class CellLocation {  
  
    private int    _row ;  
    private int    _col ;  
  
    // 기본 생성자: Cell 좌표가 주어지지 않는다  
    public CellLocation ()  
    {  
        // Cell 좌표가 주어지지 않으면 (-1, -1) 로 설정하기로 한다  
        this._row = -1 ;  
        this._col = -1 ;  
    }  
  
    // Cell 좌표가 주어지는 생성자  
    public CellLocation (int givenRow, int givenCol)  
    {  
        this._row = givenRow ;  
        this._col = givenCol ;  
    }  
  
    // 공개함수  
    .....  
}
```

□ CellLocation: 공개함수

```
public class CellLocation {  
  
    .....  
  
    // Getter / Setter  
    public void setRow (int newRow) {  
        this._row = newRow ;  
    }  
    public int row () {  
        return this._row ;  
    }  
  
    public void setCol (int newCol) {  
        this._col = newCol ;  
    }  
    public int col() {  
        return this._col ;  
    }  
} End of class "CellLocation"
```

Class “Board”



□ Class "Board"

- 마방진 판을 위한 class
- 매번 풀기 전에 차수에 맞게 새롭게 객체를 생성하여 사용한다.
 - 생성 시에 차수가 주어진다.
- 빈칸과 채워진 칸을 구분한다.
 - 빈칸은 EMPTY_CELL (-1) 값으로 채운다.
- 생성자
 - 판을 생성시키고, 모든 칸을 빈칸으로 설정한다.

	[0]	[1]	[2]
[0]	-1	-1	-1
[1]	-1	-1	-1
[2]	-1	-1	-1

실습: 마방진

□ Class Board: 생성자, 공개함수

■ 생성자

- `public Board (int givenOrder);`
 - ◆ 주어진 차수의 마방진 판을 생성한다.

■ 공개함수

- `public int order() ;`
 - ◆ 마방진 판의 차수를 얻는다.
- `public int cell (CellLocation aLocation) ;`
 - ◆ 주어진 위치 `aLocation`의 cell 값을 얻는다.
- `public int cell (int aRow, int aCol) ;`
 - ◆ 주어진 위치 (`aRow`, `aCol`) 의 cell 값을 얻는다.
- `public void setCell (CellLocation aLocation, int aCellValue) ;`
 - ◆ 주어진 값 `aCellValue` 를 주어진 위치 `aLocation`의 cell 에 넣는다.
- `public void setCell (int aRow, int aCol, int aCellValue) ;`
 - ◆ 주어진 값 `aCellValue` 를 주어진 위치 (`aRow`, `aCol`)의 cell 에 넣는다.
- `public boolean cellsEmpty (CellLocation aLocation) ;`
 - ◆ 주어진 위치의 cell 이 비어 있는지 여부를 알려준다
 - ◆ 비어 있으면 `true` 를, 아니면 `false` 를 얻는다.

Class “Board”의 구현



□ Class Board [1]

```
public class Board {  
    private static int EMPTY_CELL = -1 ;  
  
    private int      _order ;  
    private int[][]  _cell ;  
  
    // 기본 생성자  
    public Board (int givenOrder)  
    {  
        this._order = givenOrder ;  
        this._cell = new int[givenOrder][givenOrder] ;  
        for ( int row = 0 ; row < givenOrder ; row++ ) {  
            for ( int col = 0 ; col < givenOrder ; col++ ) {  
                this._cell[row][col] = Board.EMPTY_CELL ;  
            }  
        }  
    }  
}  
.....
```

Class Board [2]

```
public class Board {  
    ....  
    // 기본 생성자  
    ....  
  
    // 공개함수  
  
    // 마방진 판의 차수를 얻는다.  
    public int order() {...}  
  
    // 주어진 위치 aLocation의 cell 값을 얻는다.  
    public int cell (CellLocation aLocation) {...}  
  
    // 주어진 위치 (aRow, aCol) 의 cell 값을 얻는다.  
    public int cell (int aRow, int aCol) {...}  
  
    // 주어진 수 aNumber를 주어진 위치 aLocation의 cell 에 넣는다.  
    public void setCell (CellLocation aLocation, int aNumber) {  
        this._cell[aLocation.row()][aLocation.col()] = aNumber ;  
    }  
  
    // 주어진 수 aNumber 를 주어진 위치 (aRow, aCol) 에 넣는다.  
    public void setCell (int aRow, int aCol, int aNumber) {...}  
  
    // 주어진 위치의 cell이 비어 있는지 여부를 알려준다  
    // 비어 있으면 true, 아니면 false를 얻는다.  
    public boolean cellIsEmpty (CellLocation aLocation) {.....}  
  
} End of class "Board"
```

보고서 작성과 제출



□ 보고서 작성 방법

■ 겉장

- 제목: 자료구조 실습 보고서
- [제xx주] 숙제명 (예: [제01주] 마방진)
- 제출일
- 학번/이름

■ 내용

1. 프로그램 설명서

1. 주요 알고리즘 /자료구조 /기타
2. 함수 설명서
3. 종합 설명서

2. 프로그램 장단점 분석

3. 실행 결과 분석

1. 입력과 출력 (화면 capture 시킬 것)
2. 결과 분석

4. 소스코드

□ 과제 제출

■ 파일 이름 작명 방법

- DS1_01_학번_이름.zip

- 폴더의 구성

- ◆ DS1_01_학번_이름

- 프로그램

- 프로젝트 폴더 / 소스

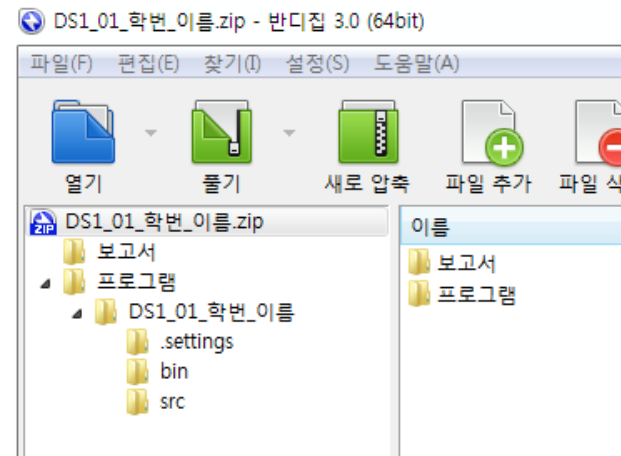
- 메인 클래스 이름 : DS1_01_학번_이름.java

- 보고서

- 이곳에 보고서 문서 파일을 저장한다.

- 입력과 실행 결과는 화면 image로 문서에 포함시킨다.

- 문서는 pdf 파일로 만들어 제출한다.



[제 1 주] 끝

**많은 것을 얻는 한 학기가
되기를!!**



